

COM106: Introduction to Databases

Entity-Relationship Modelling (Part 1 / 2)



COM106 - Introduction to Databases (S1 2627): Weekly Schedule

COM106 Weekly Schedule - Lecture & Lab Delivery

Week 9 Lab is the in-class exam (Set Exercise, 90-minute class test, 40% of module mark).

Week	Lecture	Lab / Seminar
1	Module Introduction & DB Basics and Security	Lab 1: Getting Started with MySQL
2	Introduction to Entity-Relationship Modelling - Database development stages - Entities & attribute types - Relationships: degree & cardinality - Participation constraints (full / partial) - ER diagram notation & worked example	Lab 2: Creating ER Diagrams with draw.io - ERD components & notations (Chen, Crow's Foot) - draw.io basics: shapes & connectors - Advanced draw.io features - Practice scenarios (Bookstore, Company)

<https://github.com/KodZone/COM106-MYSQL>



Introduction to Entity-Relationship Modelling

- **Entity-Relationship (ER) modelling** is a foundational method in **database design** that helps **represent real-world data and its interconnections**. It focuses on three basic components:
 - 1) Entities; 2) Attributes; 3) Relationships

Component	Definition	Examples
Entity	A distinct object, concept, or thing in the real world with a unique identity.	Student, Product, Order
Attribute	A property or characteristic that describes an entity, providing specific details.	Student: Name, ID; Product: Price, SKU
Relationship	An association between two or more entities that describes how they interact or are connected.	Student-Course Enrolment, Customer-Order Purchase

Database development stages



Database Development Stages

Stage	Purpose/Tasks	Outcome
1. Data Investigation	- Determine what data exists and how it will be used - Begin outlining the data dictionary	- Initial Data Dictionary capturing essential data definitions.
2. Data Modelling	- Shape real-world facts into a conceptual model - Identify entities, attributes, and relationships	- Generalized Conceptual Model (including relational schema).
3. Database Conceptual Design	- Convert the conceptual model into a DBMS-specific design - Define external schemas based on the chosen platform	- DBMS-Specific Conceptual Design (including external schema)
4. Database Implementation	- Create the physical database structure - Load data and define storage structures	- Internal Schema and a Populated Database
5. Security, Monitoring, and Tuning	- Implement security measures - Monitor performance, optimize, and reorganize if necessary	- Secure and Optimized Database suitable for ongoing maintenance

- The Data Investigation and Data Modelling stages are **independent of the eventual implementation** and of the DBMS used to realise it.
- Investigation and modelling are **critically important**. Errors at this stage that find their way into the developed system can be notoriously **difficult to correct**.



Terms Explanation

Term	Definition	Purpose
Initial Database Dictionary	A preliminary collection of key data elements and definitions established during the Data Investigation stage, outlining essential terms without full metadata details.	Forms a foundation for understanding data early on, guiding later detailed design and modelling efforts.
Conceptual Model	A high-level, DBMS-independent representation of entities, attributes, and relationships that models business requirements logically.	Captures the logical structure of the data, ensuring business requirements are clearly represented.
External Schema	A tailored, user-specific view of the database that presents only the data relevant to specific users or applications.	Provides simplified, secure access by showing only the necessary data for particular roles or tasks.
Internal Schema	The physical implementation details of the database, including data storage methods, file structures, and indexing strategies.	Guides the DBMS for efficient data storage, retrieval, and performance optimization in the physical system.



Data Investigation

Process	Tasks Involved	End Result
<i>Data Investigation</i>	<i>Identification of nature and use of data</i>	<i>Outline Data Dictionary</i>
-----	-----	-----

Within the constraints of the **business scenario**, the database designer must decide:

- What are the **entities**, and the **relationships** among these entities in the application?
- What information about these entities and relationships should we store in the database - **attributes**?
- What are the **integrity constraints** or **business rules** that hold?

The database designer must investigate the current system to **understand the requirements**

- Interview customers, use existing system forms/reports/common queries, etc, etc.
- Determine the **scope** of the required system – what’s important and what’s not.

Output :

An outline **Data Dictionary**: Set of **attributes** with sample values , source, usage etc.

Attributes can be grouped into **entity types** (i.e. what is the basic object that an attribute is describing)



Data Modelling

Process	Tasks Involved	End Result
-----	-----	-----
<i>Data Modelling</i>	<i>Shaping real world facts into real world concepts</i>	<i>Generalised Conceptual Model</i>
-----	-----	-----

Produce a single **conceptual model** of the system using:

Entity Relationship Modelling

top down approach (generalisation of entities/relationships)

Normalisation (We will look at this later in the module)

bottom up approach (linked to relational data model)

The next stage of the database design process involves transforming the **conceptual model** into a **data model** ready for implementation in a particular DBMS (e.g., relational, object-oriented, etc).

In our case, we will end up with a **Relational Schema**, ready for implementation in a Relational DBMS (**RDBMS**).



Generalization vs Normalization in Data Modelling

- **Generalization (Top-Down Approach):**
 - Start with a high-level view of the system by identifying general entities and relationships. Then, refine these broad concepts into detailed models.
 - *Example:* Designing a university system by first defining broad entities like **Student**, **Course**, and **Department**, and later detailing what information each entity should include.
- **Normalization (Bottom-Up Approach):**
 - Start with the detailed data you have, organize it into structured tables through normalization, and then link these tables together.
 - *Example:* Integrating existing spreadsheets for sales, inventory, and customers by creating normalized tables (such as **Sales**, **Products**, **Customers**) and establishing relationships between them.
- Designers can choose either approach—or a combination of both—depending on the project's requirements and the data available.



ER Modelling and Normalisation

The Conceptual Model developed must be **relevant**, and **strictly limited** to the application.

Consider a Private Hospital:

a database developed for **billing patients** would not need to include information about **allergies**.

Different applications can have different needs, and different perspectives – even to model the **same** object

Billing Department:

PATIENT(id, name, insurance, address)

VISIT(patientId, procedure, date, charge)

Inpatient:

PATIENT(id,name,age,address)

ALLERGIES(id,allergies)

PERScription(patientId,date,medicine)

The process of designing a database involves:

1. Creating an **ER Diagram** based on the results of a data investigation (**ER Modelling**)
2. Using the mapping (translation rules) to create a **relational schema** (a set of linked tables)
3. Checking that each table is correctly designed (i.e. in **3rd Normal Form**) which may involve decomposition (splitting into more tables)

Steps 1 & 2 are part of **ER modelling**; 3 is **Normalisation**

Entities & attribute types



ER Modelling - A Brief Overview

Entities

The basic concept of the ER Model is the **Entity Type**.

An **entity type** is a group of objects having the same properties recognised by an enterprise as:

- **important**
- uniquely identifiable
- has more than one **attribute** to describe it.

Can have a physical or 'real' existence

– PART, SUPPLIER

Or a conceptual or 'abstract' existence

– SALE, INSPECTION

Relevant to the given problem.

Information on VEHICLE may or may not be relevant in a database concerned with an enterprise's staff.

An attribute is a property of an entity type (e.g. surname)

An **Entity Occurrence** is a uniquely identifiable occurrence of an entity type.

Relationships

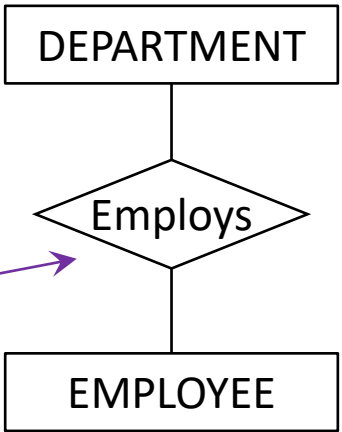
A **relationship** is a meaningful association between two (or more) entities.

e.g. The **EMPLOYS** relationship links DEPARTMENT & EMPLOYEE entity types

Entity Relational (ER) Diagrams

ENTITY TYPE: Represented by labelled rectangular box

RELATIONSHIP: Represented by labelled diamond box
between pair of entities





ER Modelling - Entities

How do we determine what the entities should be?

Start **by understanding the application** required.

- By talking with the customer/users/stakeholders to understand the application required,
- By studying the existing system/process, forms, sorts of information required, etc

Some Examples:

People: staff, clients, patients, members, owners, contacts, other individuals, . . .

Objects: stock items, real estate, vehicles, offices, . . .

Organisations: suppliers, customers, departments, charities, clubs, committees, . . .

Object classes: recordings, films, books, types of stock, biological species, roles, . . .

Events: concerts, examinations, lecture courses, consultations, sales, . . .

Potential entities are usually **nouns** in natural language - but not every noun should be an entity:

- Is there more than one instance of this entity? If not, a constant will do.
- If there are a number of instances, are these fixed? (e.g. the days of the week)? If so, there is no need to create an entity.
- Is anyone really interested in the instances? If not, there is no need for the entity.



ER Modelling - Entities and Attributes

Attribute:- A **property** of an entity type which associate each entity with a value from a **domain** of values for that attribute

(Relationship types can also have attributes - more on relationships and attributes later).

Simple Attribute:- An attribute composed of a **single component** with an **independent existence** (e.g. employee name, part number, cost, etc).

Composite Attribute:- An attribute composed of **multiple components** each with an **independent existence** (e.g. Date is day/month/year).

Multi-valued:- e.g. phone number

Derived Attribute:- An attribute with a value that is **derivable** from one or more attributes (e.g. Total Cost = Quantity * Unit Cost).

Every instance of an entity type **must** be **uniquely identifiable** (from every other instance). For example, every employee must be **identifiable as different** from any other employee.

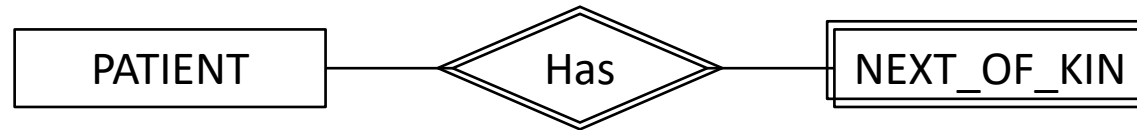
An attribute - or combination of attributes - is chosen to identify each instance of an Entity Type - this is equivalent to the **Primary Key**.



ER Modelling - Weak and Strong Entities

Weak Entity Type:- an entity type that is **existence-dependent** on some other entity type
i.e., only exists if another entity exists.

Consider a Hospital Records application:



NEXT_OF_KIN is weak in the relationship

NEXT_OF_KIN has **no independent existence** – only recorded if a **PATIENT** is recorded – no other relationship.

Strong Entity Type:- An entity type that is **not existence-dependent** on some other entity type.

EMPLOYEE and **DEPARTMENT** are examples

Most entity types are strong

Relationships: degree & cardinality



ER Modelling - Relationships

Some definitions:

Degree of a Relationship (**unary, binary, ternary, etc**)

The number of participating entity types in a relationship.

Cardinality Ratio (**1:1, 1:N, N:1, M:N**)

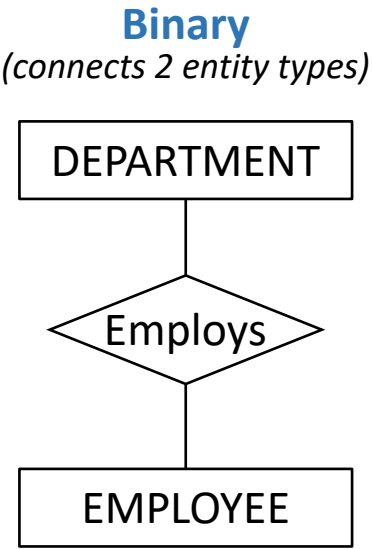
Determines the type of relationships between the participating entity types.

Participation Constraints (**Full or Partial**)

Determines whether the existence of an entity type depends on its being related to another entity type through the relationship

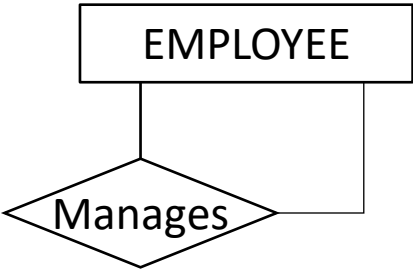


ER Modelling - Degree of a Relationship



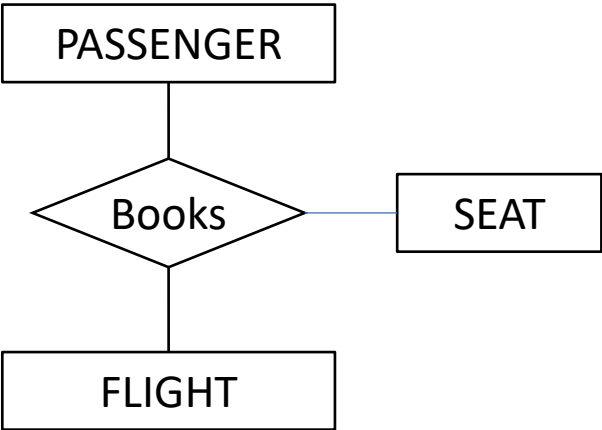
Unary Relationship (Degree 1):
A relationship where an entity is related to itself.
*Example: An **Employee** entity may have a "supervises" relationship with other employees. Here, one employee supervises another within the same entity type.*

Unary or Recursive
(connects one entity type with itself)



Binary Relationship (Degree 2):
A relationship involving two different entity types.
*Example: A **Customer** places an **Order**. This relationship connects the Customer entity with the Order entity.*

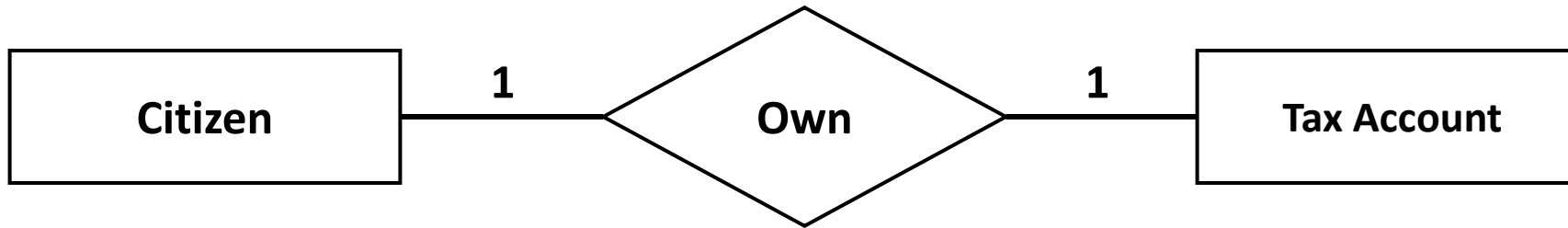
Complex
(connects three or more entity types
(Ternary - connects 3))



Ternary Relationship (Degree 3):
A relationship that involves three entity types.
*Example: Consider a scenario involving **Supplier**, **Product**, and **Store**. A ternary relationship might indicate that a Supplier provides a Product to a specific Store.*

1:1 Cardinality Ratio

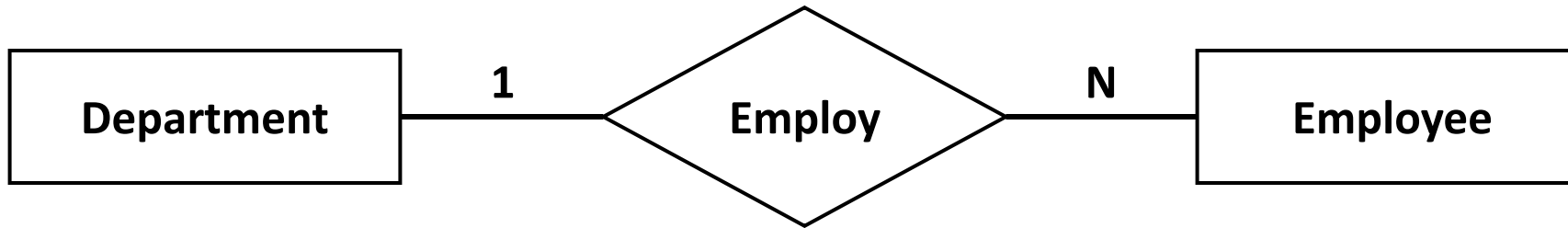
- 1:1
 - **Every** (Entity A) Relationship **at most one** (Entity B), AND
 - **Every** (Entity B) Relationship **at most one** (Entity A)



- **Every** Citizen own **at most one** Tax Account AND **Every** Tax Account owned by **at most one** Citizen.

1:N Cardinality Ratio

- 1:N
 - **Some** (Entity A) **may** Relationship **many** (Entity B),
 - **AND Every** (Entity B) Relationship **at most one** (Entity A)

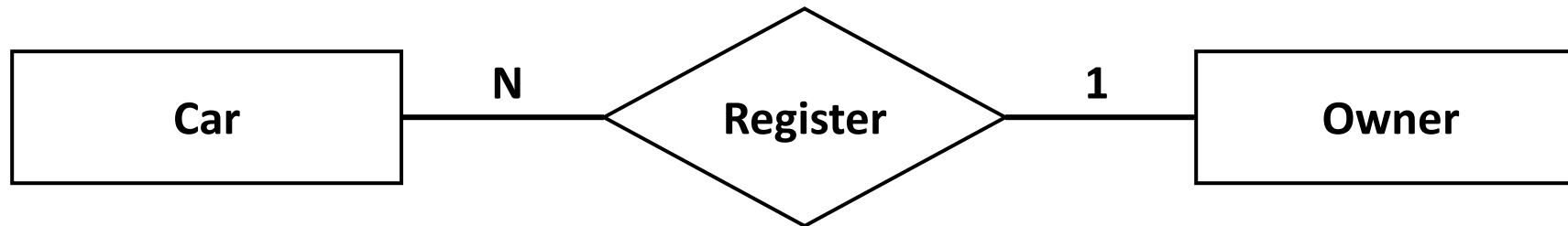


- **Some** Departments employ **many** Employees **AND Every** Employee employed by **at most one** department.



N:1 Cardinality Ratio

- N:1
 - **Every** (Entity A) Relationship **at most one** (Entity B),
 - AND **Some** (Entity B) **may** Relationship **many** (Entity A).

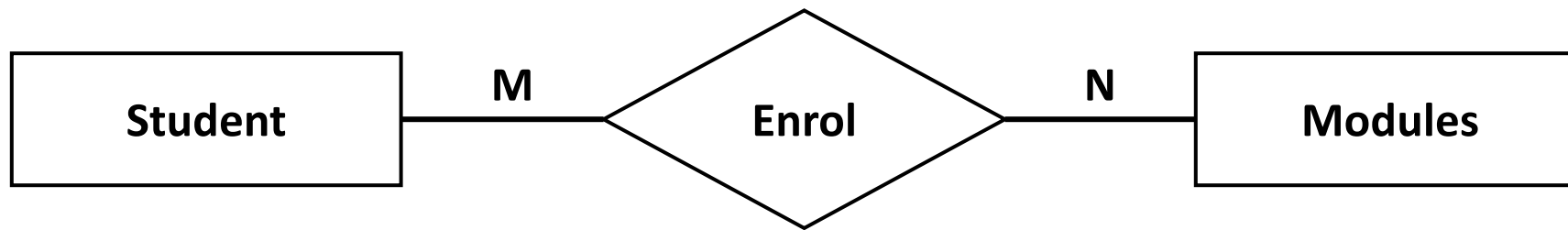


- **Every** Car registered by **at most one** Owner AND **some** Owners **may** register **many** Cars.

N:M Cardinality Ratio

- N:M

- **Some** (Entity A) **may** Relationship **many**(Entity B),
- **AND Some** (Entity B) **may** Relationship **many** (Entity A).



- **Some** Students **may** enrol **many** Modules **AND Some** Modules **may** enrolled by **many** students .



Cardinality Ratio of a Relationship - Examples

. **One-to-One (1:1)** - Each instance of Entity A is associated with one instance of Entity B, and vice versa.

- **Person – Passport**

- Each person has exactly one passport, and each passport is issued to exactly one person.

One-to-Many (1:N) - One instance of Entity A can be associated with multiple instances of Entity B, but each instance of Entity B is associated with only one instance of Entity A.

- **Employee – Department**

- Many employees can belong to one department. (This is the same relationship as above but viewed from the employees' perspective.)

Many-to-Many (M:N) - Multiple instances of Entity A can be associated with multiple instances of Entity B.

- **Student – Course**

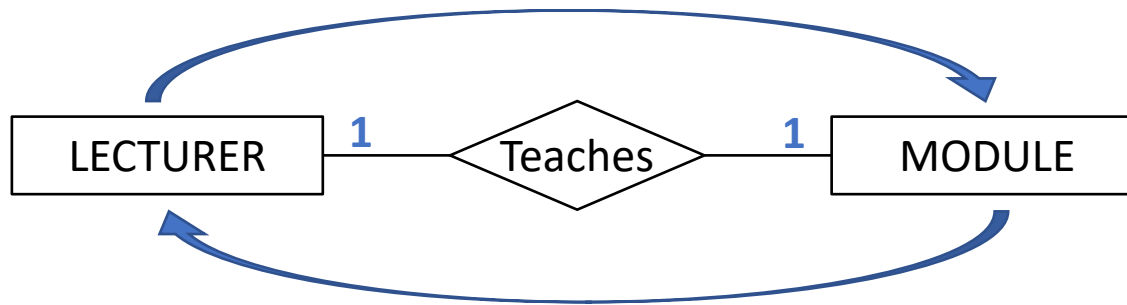
- A student can enroll in multiple courses, and each course can have many students enrolled.



ER Modelling - Cardinality Ratio of a Relationship

To determine the cardinality ratio, you must consider the relationship in **BOTH** directions

Consider the following example:



Does every Lecturer teach at most one or more than one (many) module?

Is every Module taught by at most one or more than one Lecturer?

1:1 (one-to-one)

True for the **Teaches** relationship if the following rules exist:

- Every Lecturer teaches **at most** one Module
(1:1 in the direction LECTURER to MODULE)
- Every Module is taught by **at most** one Lecturer
(1:1 in the direction MODULE to LECTURER)

Overall a 1:1 relationship is a combination of two 1:1 relationships in both directions



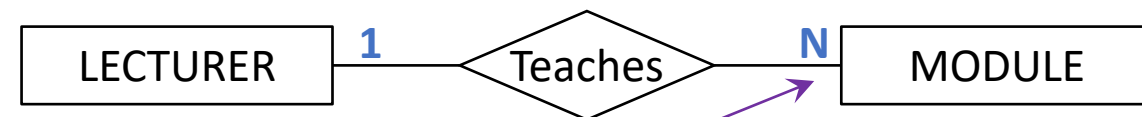
ER Modelling - Cardinality Ratio of a Relationship

1:N (one-to-many)

True for the **Teaches** relationship if the following rules exist:

- Some Lecturers may teach **more than one (many)** Modules *(although other might teach only one)*
- All Modules are taught by **at most** one Lecturer

So a 1:N relationship is made up of a **1:N** relationship in one direction
AND a **1:1** relationship in the other



*In the business scenario:
nouns can suggest **entities**,
verbs can suggest **relationships***

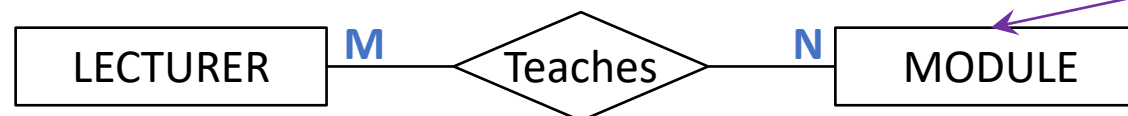
*By convention, entity names
are singular – **MODULE**
rather than **MODULES***

M:N (many-to-many)

True for the **Teaches** relationship if the following rules exist:

- Some *(not necessarily all)* Lecturers may teach **many (more than one)** Modules
- Some *(not necessarily all)* Modules are taught by **many (more than one)** Lecturers

Overall a M:N relationship is a combination of **two 1:N relationships** in both directions

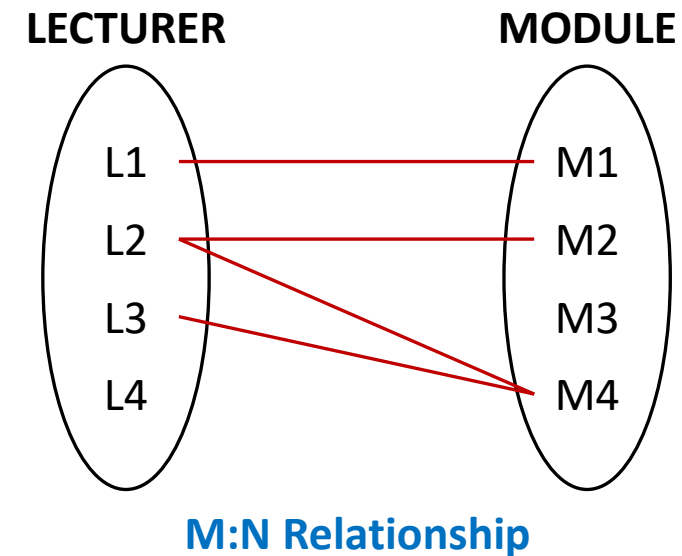
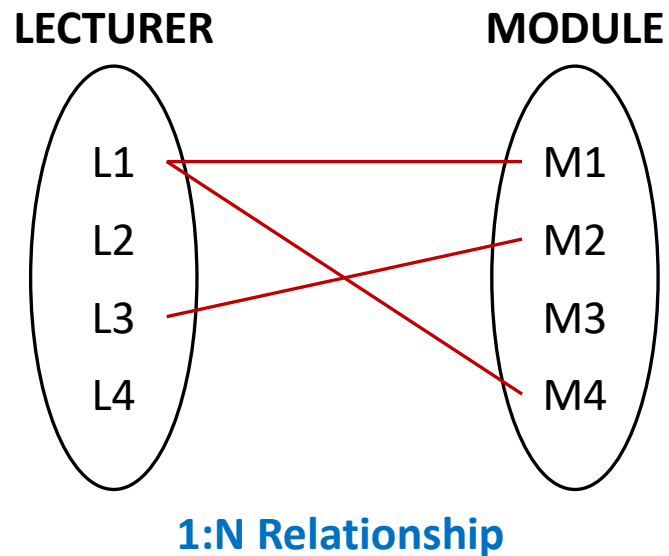
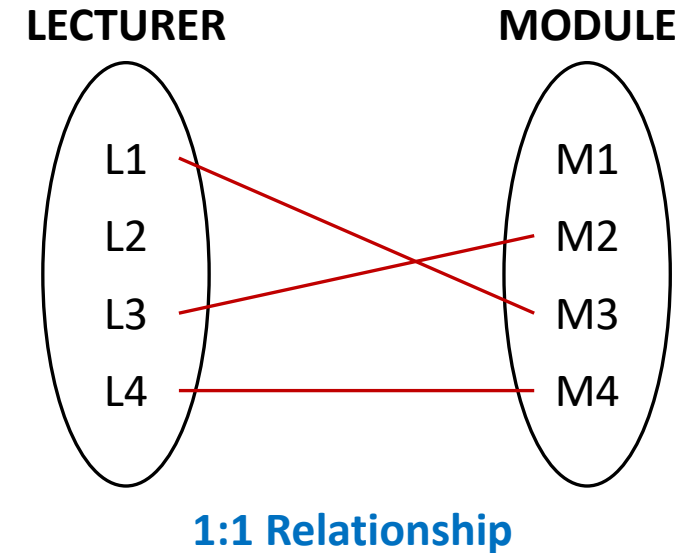


Participation constraints (full / partial)

Cardinality Ratio of a Relationship - Semantic Networks

A **semantic network**, is a common tool used to represent **meaningful relationships** between **concepts**.

Here they are used to represent **relationships** between **entity occurrences**.





Relationship Cardinality – Time (Current vs. Historical)

Often the time requirements of a relationships affect its cardinality

For the relationship **DEPARTMENT** **Employs** **EMPLOYEE**

- if the database wants to record only the **CURRENT** situation (i.e. which Department employs which Employee **at the moment**), then a **1:N relationship is appropriate**.

Thus, if an employee changes department the record of his old department is **deleted**

- if however the database wants to record the **HISTORIC** situation (i.e. which departments the employee has been in, **including past departments**), then the **appropriate relationship is M:N**.

Thus, records of an employee's old departments are **stored and not deleted**.



ER Modelling - Participation Constraints

The **participation constraint** is a characteristic of how an entity type participates in a particular relationship.

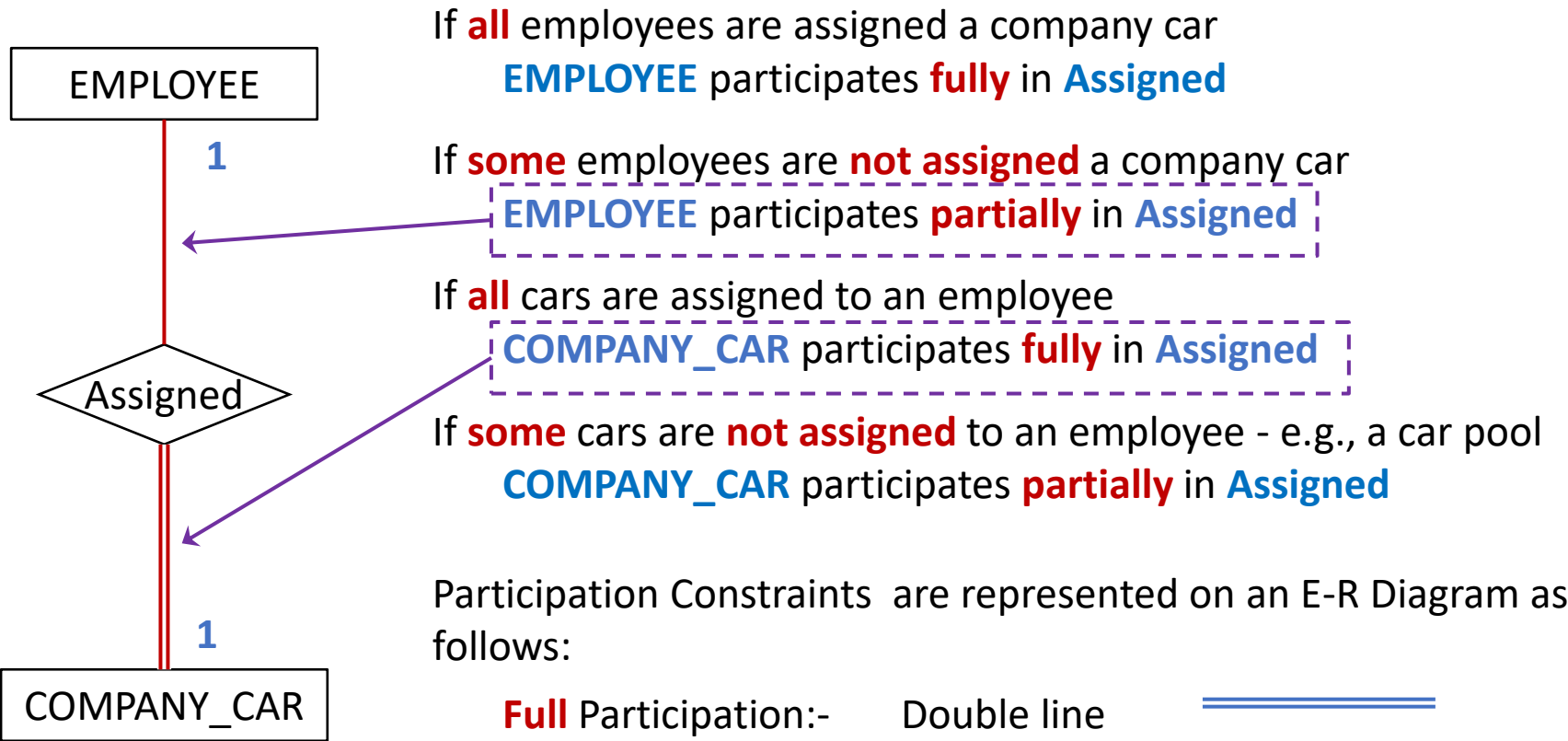
The questions to be asked are:

- 1. Is every instance of an entity type linked to an instance of the other entity type in the relationship?**
(full participation aka mandatory participation – no instances of the entity type are allowed to exist unless they are involved in the relationship)
- 2. Or, are some instances not linked?** They exist but are not participating in the relationship
(partial participation aka optional participation – some instances of the entity type can exist without involvement in the relationship)



Participation Constraints – An Example

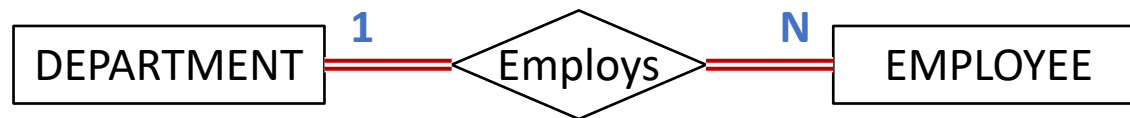
Consider an E-R Diagram showing a 1:1 relationship (**Assigned**) between **EMPLOYEE** and **COMPANY_CAR**



So – if **not all** employees are allocated car and **all** cars are allocated to some employees



ER Modelling - Participation Constraints



Consider the **DEPARTMENT** **Employs** **EMPLOYEE** relationship:

- if **all instances** of the **EMPLOYEE** entity type (i.e. all employees) participate in the **Employs** relationship (i.e. are employed in some department) then **participation constraint** of **EMPLOYEE** in **Employs** is **FULL**
- if however, **any instances** of **EMPLOYEE** (i.e. any employees) do not participate in the **Employs** relationship (i.e. exist but not assigned to any department) then the **participation constraint** of **EMPLOYEE** in **Employs** is **PARTIAL** (**Rationally EMPLOYEE in Employs is FULL**)
- There are always **two** possible participation constraints in a (binary) relationship (i.e. **EMPLOYEE** in **Employs** and **DEPARTMENT** in **Employs**) and these **don't have to be the same**



ER diagram notation & worked example

Project Management System

(Entities – Relationships - Degree of a Relationship -
Cardinality Ratio - Participation Constraints – Attributes)



Project Management System: 1) Entities

Draw an E-R Diagram which models a **Project Management**, where Each DEPARTMENT employs EMPLOYEES, who work in various PROJECTS where different PARTS are used.

Timescale: Current time

1) Identify the Entity Types

Each DEPARTMENT employs EMPLOYEES, who work in various PROJECTS where different PARTS are used.

DEPARTMENT

EMPLOYEE

PARTS

PROJECT



Project Management System: 2) Relationships

Each DEPARTMENT employs EMPLOYEES, who work in various PROJECTS where different PARTS are used.

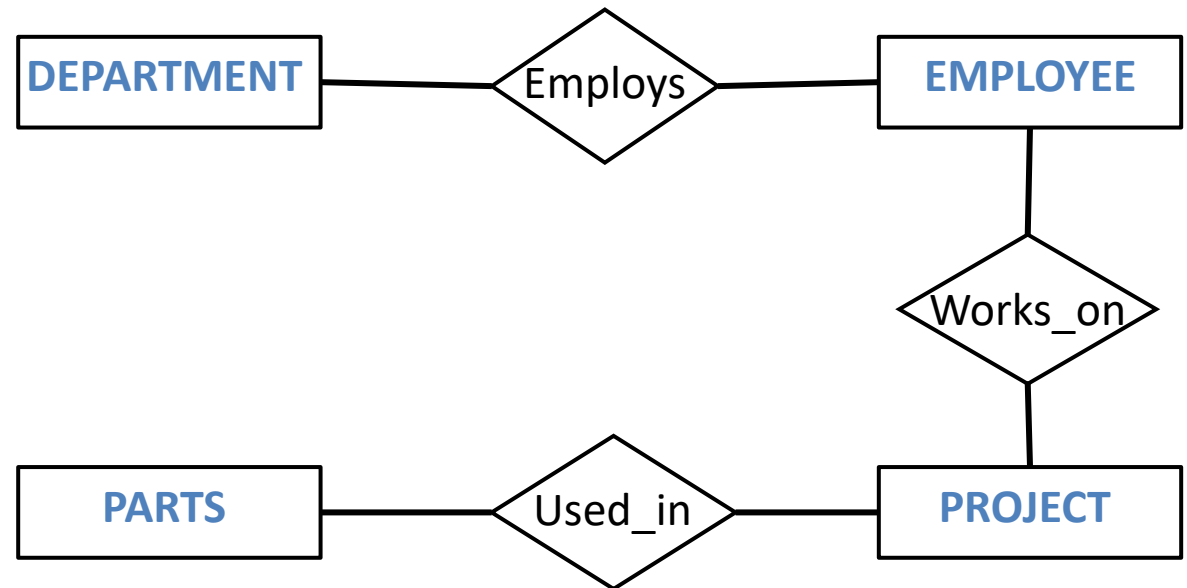
2) Identify the Relationships between Entity

Most Important:

DEPARTMENT **Employs** EMPLOYEE

EMPLOYEE **Work_on** PROJECTS

PROJECTS **USED** PARTS



Project Management System: 3) Cardinality Ratios

Each DEPARTMENT employs EMPLOYEES, who work in various PROJECTS where different PARTS are used.

3) Determine the **Cardinality Ratios** of each relationship

Consider the **Employs** Relationship (*in both directions*)

- Does every **DEPARTMENT** employ at most one or more than one **EMPLOYEE**?

1 to many – a department normally employs many employees

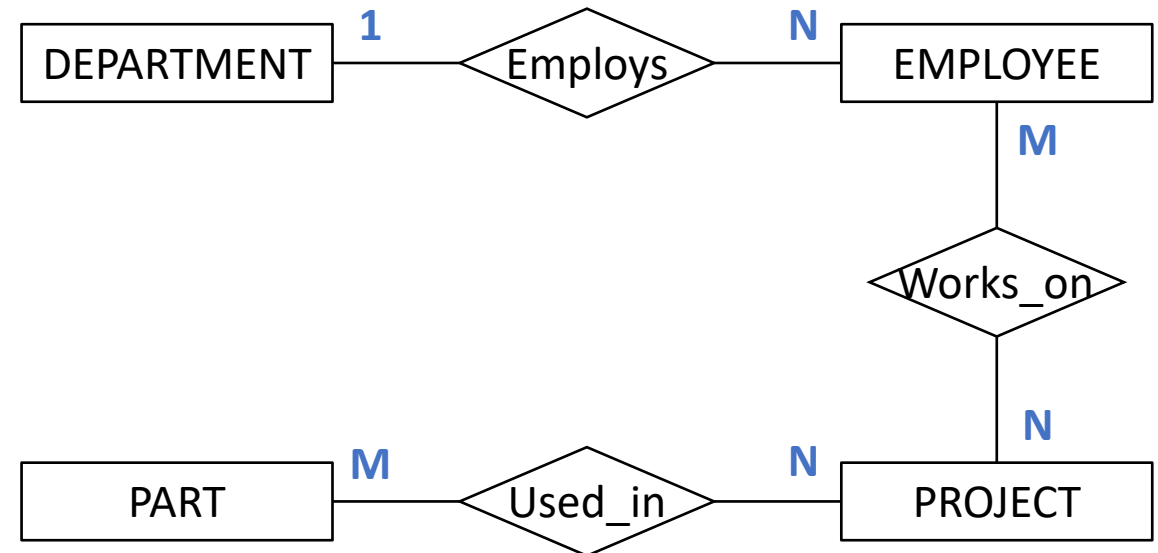
- Is every **EMPLOYEE** employed in just one **DEPARTMENT**

1 to 1 - an employee is normally employed in just one department *assuming we are considering their current situation only*.

So, overall, the **Employs** relationship between **DEPARTMENT** and **EMPLOYEE** has a cardinality ratio of **1:N**.

Now consider the **Used_in** relationship

And finally, the **Works_on** relationship





Project Management System: 4) Participation Constraints

Each DEPARTMENT employs EMPLOYEES, who work in various PROJECTS where different PARTS are used.

3) Determine the **Participation Constraints** of each relationship

A participation constraint is how an entity type participates in a relationship (**FULL/PARTIAL**)

DEPARTMENT in **Employs** relationship

FULL - if every department is linked to at least 1 employee

PARTIAL - if some departments exist with NO employees

EMPLOYEE in **Employs** relationship

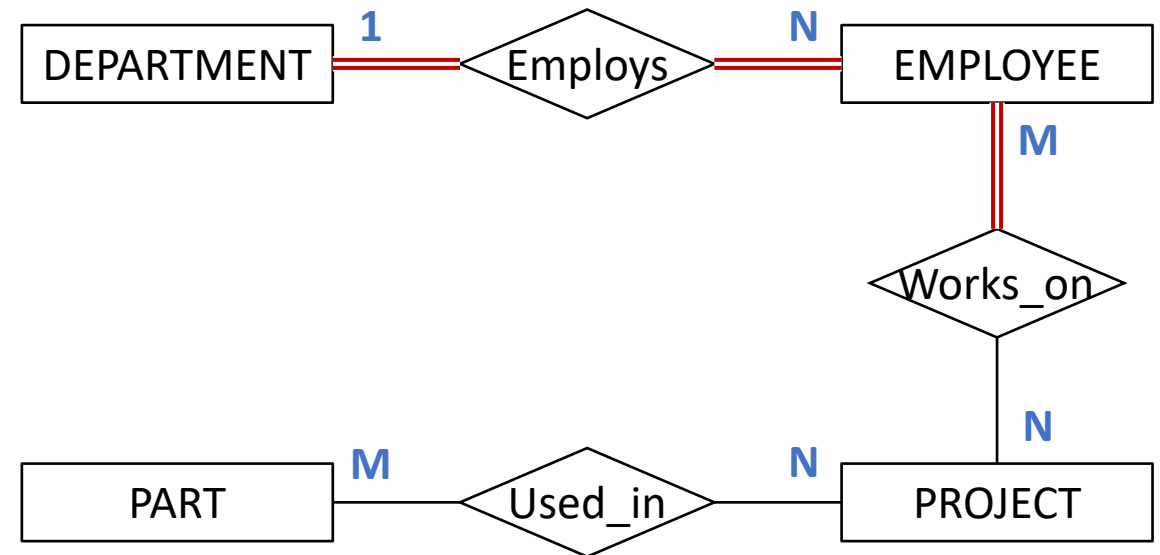
FULL - if every employee is always in one department

PARTIAL - if some employees exist with NO department

EMPLOYEE in **Works_on** relationship

FULL - if every employee has to work on at least one project

PARTIAL - if some employees work on NO projects





Project Management System: 4) Participation Constraints

Example continued:

PROJECT in **Works_on** relationship

FULL - if every project has at least one employee working on it

PARTIAL - if some projects can exist with NO employees working on them

PROJECT in **Used_in** relationship

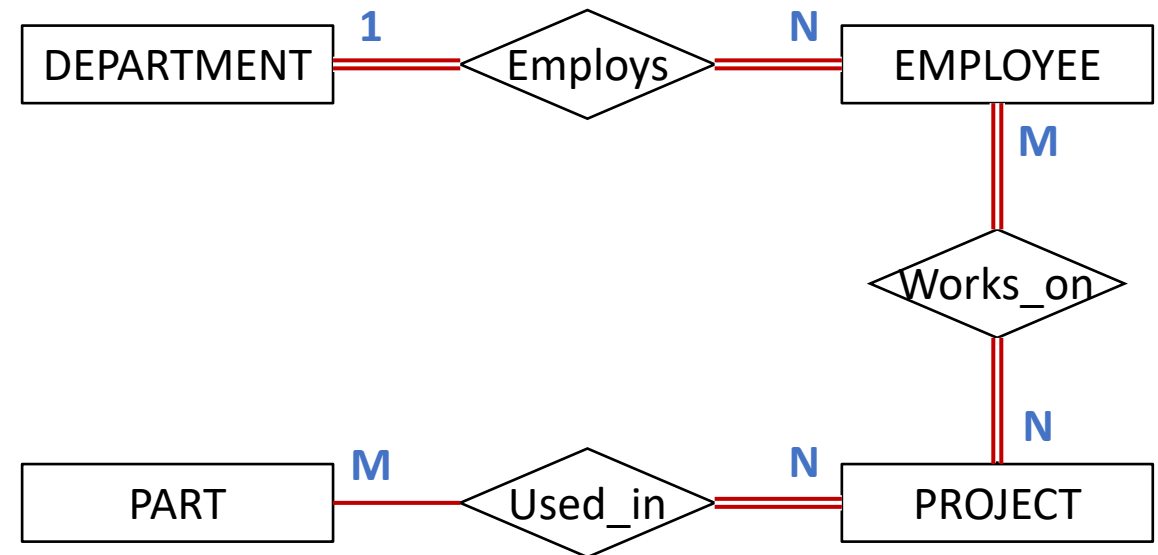
FULL - if every project uses at least one part

PARTIAL - if some projects can exist using NO parts

PART in **Used_in** relationship

FULL - if every part is used in at least one project

PARTIAL - if some parts exist but are not used in any project



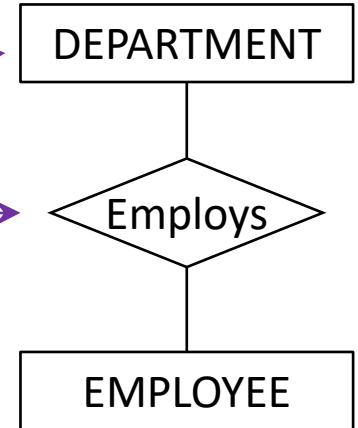


Conclusion

- **Entity-Relationship (ER) modelling** is a foundational method in **database design**, in which **entity** is a unique identity of object, concept, or thing in the real world that described by **attributes** and connected to other entities by **Relationship**.

- **Entity Relational (ER) Diagrams:**

- ENTITY TYPE: Represented by labelled rectangular box
- RELATIONSHIP: Represented by labelled diamond box between pair of entities



- **In ER Modelling, need to identify the following:**
 - Degree of a Relationship (unary, binary, ternary, etc)
 - Cardinality Ratio (1:1, 1:N, N:1, M:N)
 - Participation Constraints (Full or Partial)



**THE
END**